

1.2 Julia Programming Basic

In this notebook, I will go through some very basic things about Julia programming. Note that this is not meant to be a comprehensive introduction to Julia. I am just covering what I need you to know to understand the basic code this course. I will introduce new Julia elements as we go along.

For a more comprehensive explanation of Julia, see <https://julialang.org/>. If you are familiar with programming, you would have seen many of the material below before. In terms of syntax, you will find that Julia shares a lot of similarity with MATLAB.

Types

As with many programming languages, variables in Julia can have different types. By default, integers have type *Int64*, and float is double precision *Float64*. There is also a string type to represent a string. You can set values to variables in Julia by just using the = sign

```
In [13]: x=1
         y=2.0
         z="Here is a string"
```

"Here is a string"

You can find out the type of the variables by using the `typeof()` function in Julia

```
In [14]: typeof(x)
```

Int64

Note that when we set `x` to the value of 1, Julia by default set `x` to be an integer of type *Int64*

Exercise Ex01: Use `typeof()` to find out the type of variables `y` and `z`

You can force a variable to be single precision floating point with the code below

```
In [15]: p=1.0f0
         q=Float32(1.0)
         typeof(p),typeof(q)
```

(Float32, Float32)

You can group variables together by using a `tuple` in Julia

```
In [16]: t=(x,y,z)
```

(1, 2.0, "Here is a string")

Here the tuple `t` has three elements an integer, float and a string. You can also use the `typeof()` function to find out the type of variables in `t`

```
In [17]: typeof(t)
```

```
Tuple{Int64, Float64, String}
```

Note that tuples are immutable, this means that their values are set and cannot be changed after the creation of the tuple. You can access each element of the tuple by using the square bracket `[]` with a number indicating the position of the element in the tuple.

```
In [18]: t[3]
```

```
"Here is a string"
```

Another way of grouping different variables together is by using a `struct`.

```
In [19]: struct House
           Address::String
           Size::Float32
           Construction::String
       end

       MyHouse=House("Australia",100.0,"Brick")
```

```
House("Australia", 100.0f0, "Brick")
```

You can access each element of the struct by using the `.` notation

```
In [20]: MyHouse.Address
```

```
"Australia"
```

By default structs are immutable, that means you cannot change the variables in struct. The command

```
In [21]: MyHouse.Size=200
```

```
setfield!: immutable struct of type House cannot be changed
```

```
Stacktrace:
```

```
[1] setproperty!(x::House, f::Symbol, v::Int64)
```

```
@ Base ./Base.jl:53
```

```
[2] top-level scope
```

```
@ ~/Library/CloudStorage/OneDrive-TheUniversityofMelbourne/JuliaPrograms/PekingUniGlobexCourse/Chapter01Introduction/jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_X30sZmlsZQ==.jl:1
```

If you want to be able to change the elements inside struct, you will need to use a mutable struct

```
In [22]: # Define a mutable struct called Counter
mutable struct Age
    Years::Int
end
MyAge=Age(70)
```

Age(70)

```
In [23]: MyAge.Years=60
```

60

```
In [24]: MyAge.Years
```

60

Vectors, Matrix, Arrays

In scientific computing, you will be dealing mostly with arrays and matrices. Vectors are usually 1 dimensional, Matrices are two dimensional and Arrays are more general N dimensional.

You can make a undefined vector and matrix of 4 and 2x4 elements respectively

```
In [25]: Vector{Float64}(undef, 4)
```

4-element Vector{Float64}:
 2.2698151176e-314
 2.130317216e-314
 5.0e-324
 2.38806505e-314

```
In [26]: Matrix{Float64}(undef, 2, 4)
```

2×4 Matrix{Float64}:
 1.93e-322 0.0 0.0 0.0
 0.0 0.0 0.0 0.0

You can specify a column vector in Julia by using a [] bracket and separating each element by a comma

```
In [27]: a=[1.0, 2.0 ,3.0]
```

3-element Vector{Float64}:
 1.0
 2.0
 3.0

a is a column vector with 3 elements $\vec{a} = \begin{Bmatrix} 1 \\ 2 \\ 3 \end{Bmatrix}$

Exercise Ex02: Type in `a=[1 2 3 4]` . What does Julia give you?

Matrices can be defined using [], with each row of the matrices separated by a semicolon.

```
In [28]: b=[2.0 3.0 4.0;8.0 9.0 4.0]
```

2×3 Matrix{Float64}:

```
2.0  3.0  4.0
8.0  9.0  4.0
```

b is a 2×3 matrix of type *Float64* $[b] = \begin{bmatrix} 2.0 & 3.0 & 4.0 \\ 8.0 & 9.0 & 4.0 \end{bmatrix}$

You can slice rows and columns of a matrix by using : Note that for Julia, slicing a matrix will give you column vectors

```
In [29]: b[1,:]
```

3-element Vector{Float64}:

```
2.0
3.0
4.0
```

$$b[1,:]=\begin{Bmatrix} 2.0 \\ 3.0 \\ 4.0 \end{Bmatrix}$$

```
In [30]: b[:,2]
```

2-element Vector{Float64}:

```
3.0
9.0
```

$$b[:,2]=\begin{Bmatrix} 3.0 \\ 9.0 \end{Bmatrix}$$

Note that after slicing the b matrix, you will get vectors.

Exercise Ex03: Type in `b[:]`. What does Julia give you?

The transpose of a matrix can be obtained by using the `transpose()` function

```
In [31]: transpose(b)
```

3×2 transpose(::Matrix{Float64}) with eltype Float64:

```
2.0  8.0
3.0  9.0
4.0  4.0
```

The adjoint of a matrix can be obtained by putting ' after the matrix. The adjoint is the transpose and the complex conjugate of every elements of a matrix. For a matrix consisting of real elements, the adjoint would be the same as the transpose of the matrix

```
In [32]: b'
```

3×2 adjoint(::Matrix{Float64}) with eltype Float64:

```
2.0  8.0
3.0  9.0
4.0  4.0
```

You can define an array from 0 to 1 in steps of 0.1 by using a `range` using the following syntax

```
In [33]: x=0.0:0.1:1.0
```

```
0.0:0.1:1.0
```

Note that this is NOT an array. A `range` is a sequence of numbers BUT it does not allocate memory to every element in the sequence of numbers. This is to save memory if you are generating a large sequence of numbers. To convert a `range` into an `Array`, use the `Array()` function in Julia

```
In [34]: Array(x)
```

```
11-element Vector{Float64}:
```

```
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0
```

You can use the code below to generate generate a matrix of values for a range of x and y . This method is called "array comprehension" and it is a compact way of generating large matrices. The example below generates the matrix

$$\begin{bmatrix} x_1 + y_1^2 & x_1 + y_2^2 & x_1 + y_3^2 \\ x_2 + y_1^2 & x_2 + y_2^2 & x_2 + y_3^2 \\ x_3 + y_1^2 & x_3 + y_2^2 & x_3 + y_3^2 \end{bmatrix}$$

from the 1d vectors

$$x = \begin{Bmatrix} -1 \\ 0 \\ 1 \end{Bmatrix} \text{ and } y = \begin{Bmatrix} 1 \\ 2 \\ 3 \end{Bmatrix}$$

```
In [35]: xrange=-1:1:1
          yrange=1:1:3
          loss_values=[x+y^2 for x in xrange, y in yrange]
```

```
3×3 Matrix{Int64}:
```

```
0  3  8
1  4  9
2  5 10
```

$$\text{loss_values} = \begin{bmatrix} 0 & 3 & 8 \\ 1 & 4 & 9 \\ 2 & 5 & 10 \end{bmatrix}$$

Operators

The normal operators apply in Julia if the left and right arguments are scalar

```
In [36]: a=1.0+2.0 # addition
         b=1.0-2.0 #subtraction
         c=2.0*3.0 #multiplication
         d=3.0^3 #power

         (a,b,c,d)
```

```
(3.0, -1.0, 6.0, 27.0)
```

There is a difference between the forward slash / and backslash \ operators in Julia

```
In [37]: 1.0/2.0

0.5
```

```
In [38]: 1.0\2.0

2.0
```

Let's now see what happens if the arguments to the operators are vectors/matrices.

The normal rules apply

```
In [39]: a=[1,2,3]
         b=[2.0 3.0 4.0;8.0 9.0 4.0]
         b*a
```

```
2-element Vector{Float64}:
 20.0
 38.0
```

a is a 1×3 vector, b is a 2×3 matrix,

$$\vec{a} = \begin{Bmatrix} 1 \\ 2 \\ 3 \end{Bmatrix} [b] = \begin{bmatrix} 2.0 & 3.0 & 4.0 \\ 8.0 & 9.0 & 4.0 \end{bmatrix}$$

$b|a^*$ will give you a 2×1 matrix

$$b*a = \begin{bmatrix} 2.0 & 3.0 & 4.0 \\ 8.0 & 9.0 & 4.0 \end{bmatrix} \begin{Bmatrix} 1 \\ 2 \\ 3 \end{Bmatrix} = \begin{Bmatrix} 20 \\ 38 \end{Bmatrix}$$

You can multiply matrix with a vector

In [50]: `b*a`

2-element Vector{Float64}:
20.0
38.0

Exercise Ex04: Does `a*b` or `a*a` work? Why?

We can perform elementwise operations by preceding the operator with a `.`

In [52]: `a.*a`

3-element Vector{Int64}:
1
4
9

In [54]: `b.*b`

2×3 Matrix{Float64}:
4.0 9.0 16.0
64.0 81.0 16.0

In [55]: `a.*b[1,:]`

3-element Vector{Float64}:
2.0
6.0
12.0

Exercise Ex05: What does `a.*b[1,:]` give you? Can you explain why?

When you have vectors and matrices, the `\` operator solve the linear system of equations defined by the matrices and vectors

$$\mathbf{A}\vec{x} = \vec{C} \quad (1)$$

$$\vec{x} = \mathbf{A}^{-1}\vec{C} \quad (2)$$

In [56]: `A=[1 2 3;4 5 6;7 8 9]`
`C=[2, 3, 4]`
`x=A\C #equivalent to A^{-1}*C`
`A*x-C`

3-element Vector{Float64}:
0.0
0.0
8.881784197001252e-16

Functions

Functions are used to group codes that is repeatedly used. Simple functions in Julia can be defined by a one line syntax.

In [42]: `f(x)=x^3`

`f` (generic function with 1 method)

```
In [43]: typeof(f(1)),typeof(f(2.0))
```

```
(Int64, Float64)
```

Note that the type of the return value of f depends of the input argument type, x . If x is an integer, then f returns an integer. If x is an float, then f returns an float.

You can make `f` act on every element of `a` by using the `.` operator

```
In [44]: f.(a)
```

```
3-element Vector{Int64}:
```

```
 1
 8
27
```

Exercise Ex06: What does `f(a)` give you? Why?

More complex functions can be defined by using the `function` keyword. The last expression in the function is the return value by default

```
In [45]: function my_first_function(x,y)
           mean=0.5*(x+y)
       end
```

```
my_first_function(1.0,2.0)
```

```
1.5
```

One can define a function of two variables and then map it to a function of one variable. This basically ignores the other parameter. This is common in the optimization packages in Julia which will be used in future notebooks.

In the example below. `dummy_func()` is declared as function of two variables `x` and `p`. But it returns $f(x) = x^2$, i.e. only `x` is used and `p` is ignored.

```
In [46]: dummy_func(x,p)->f(x)
           dummy_func(3,10)
```

```
27
```

Scope of variables

Usually variables inside functions are local

```
In [47]: x=2
           function my_print(x)
               println(x)
           end

           my_print(3)
```


3

In the code above, the variable `x` is local inside the `my_print()` function

However, if Julia does not see a variable inside a function, then it will assume that the variable is global and try to find the value of the variable outside the function.

```
In [48]: x=2
function my_print(b)
    println(x)
end

my_print(10)
```

2

In the code above, `x` is not defined inside `my_print()` so when `println()` will find try to find the variable `x` outside the function which has a value of 2.

Loops

We usually use loops to do repeated calculations. For example the `for` loop below takes away 1 from the variable `x` 5 times

```
In [74]: x=7.0
for i=1:5
    x-=1.0
end
x
```

2.0

If you declare a variable inside the loop, then it is only available inside the loop.

```
In [54]: xoutsideloop=3.0

for i=1:3
    xoutsideloop+=1.0
    xinsideloop=xoutsideloop+1.0
end
```

```
In [55]: xoutsideloop
```

6.0

Once you finish executing the loop, the program outside the loop does not know anything about `xinsideloop` which was declared inside the for loop.

```
In [56]: xinsideloop
```

```
UndefVarError: `xinsideloop` not defined in `Main`
```

```
Suggestion: check for spelling errors or missing imports.
```

- Prepared Andrew Ooi 2nd July 2026